



GENERALITAT
VALENCIANA

Conselleria d'Educació,
Universitats i Ocupació

ceedcv

CENTRE ESPECÍFIC
D'EDUCACIÓ A DISTÀNCIA DE
LA COMUNITAT VALENCIANA

Unitat 07 — Algorítmica I

Bloc 2

1r CFGS DAW · Programació

Sergi García Barea



En aquesta sessió

Ordenar com un professional i mesurar sense cronòmetre

- 1 **Ordenació bombolla:** comparar veïns i intercanviar.
- 2 **Ordenació per inserció:** la mà de cartes del pòquer.
- 3 **Big O:** la taxa de creixement, no el cronòmetre.
- 4 **Triar l'algoritme adequat:** la taula de decisió.
- 5 **Reptes:** binària i bombolla a mà, l'EL LIO i el repàs final.



Necessites el Bloc 1: algoritme, cerca lineal, cerca binària i $O(n)$ vs $O(\log n)$.

Ordenació bombolla: els grans pugen

La bombolla compara **parelles veïnes**: si el de l'esquerra és major, els intercanvia. En cada passada, el major restant «puja» al final com una bombolla en una copa.

Bombolla.java – el nucli

```
for (int i = 0; i < n - 1; i++) {  
    for (int j = 0; j < n - 1 - i; j++) {  
        if (array[j] > array[j + 1]) {  
            int temp = array[j];  
            array[j] = array[j + 1];  
            array[j + 1] = temp;  
        }  
    }  
}
```

 Per això el bucle interior arriba només fins a `n - 1 - i`: després de cada passada, el major dels que queden ja va quedar col·locat al final.

La bombolla, dibuixada

Bombolla: comparar veïns i intercanviar — el gran “puja” al final



pas 1: 5 vs 2 → intercanvia



pas 2: 5 vs 9 → no



pas 3: 9 vs 1 → intercanvia



Fi de la passada: el 9 ja és al seu lloc

En cada passada, el major restant queda col·locat al final → el bucle interior arriba fins a $n - 1 - i$

✓ 💡 Primera passada sobre {5, 2, 9, 1} : el 9 — el major — ja és en el seu lloc. Després el 5, després el 2, després l'1.

L'optimització del flag

●●● El flag hiHaIntercanvi

```
boolean hiHaIntercanvi;  
for (int i = 0; i < n - 1; i++) {  
    hiHaIntercanvi = false;  
    for (int j = 0; j < n - 1 - i; j++) {  
        if (array[j] > array[j + 1]) {  
            // intercanvi...  
            hiHaIntercanvi = true;  
        }  
    }  
    if (!hiHaIntercanvi) break;  
}
```

 Si en una passada completa **no intercanviem res**, l'array ja està ordenat: `break` i fora.

 No millora el pitjor cas (array invertit), però converteix el **millor cas en $O(n)$** : una sola passada de comprovació.

  **Don Tip:** el patró del flag («marca si ha passat alguna cosa; si no, para») apareix en moltíssims algorismes reals. T' farà semblar sènior encara que només portes quatre unitats.

Per què la bombolla és tan lenta?

Dos bucles anidats

```
for (int i = 0; i < n; i++) {  
    for (int j = 0; j < n; j++) {  
        //  $n \times n = n^2$  operacions  
    }  
}
```

Mida	Operacions ~
10 elements	100 (bé)
1.000 elements	1.000.000 (comença a doldre)
1.000.000 elements	10^{12} (el PC demana la jubilació)



La bombolla només s'usa en dos casos: (1) estàs aprenent, i (2) l'array tindrà **menys de ~50 elements**. Per a tot lo demés, la U08.



Be the Code: la bombolla curta

BombollaCurta.java

```
int[] dades = {3, 1, 2};  
// passades fixes: només 2  
for (int i = 0; i < 2; i++) {  
    for (int j = 0;  
        j < dades.length - 1 - i; j++) {  
        if (dades[j] > dades[j + 1]) {  
            int temp = dades[j];  
            dades[j] = dades[j + 1];  
            dades[j + 1] = temp;  
        }  
    }  
}  
  
// imprimim les dades
```

Sense executar: què imprimix?

a) 3 1 2 · b) 1 2 3 · c) 1 3 2 · d) 2 1 3

El b. Passada 1: $3 \leftrightarrow 1 \rightarrow 1 \ 3 \ 2$; $3 \leftrightarrow 2 \rightarrow 1 \ 2 \ 3$. Passada 2: $1 \leftrightarrow 2 \rightarrow$ res. Eixida: 1 2 3.

Este codi no té flag: en un array ja ordenat faria totes les passades igualment. Ahí guanya la versió amb `break`.

Ordenació per inserció: la mà de cartes

Com ordenes cartes al pòquer? No ho tires tot i comences de zero: vas col·locant cada **carta nova en el seu buit** dins de la mà que ja tens ordenada. Doncs això, però en Java.

●●● Insercio.java – el nucli

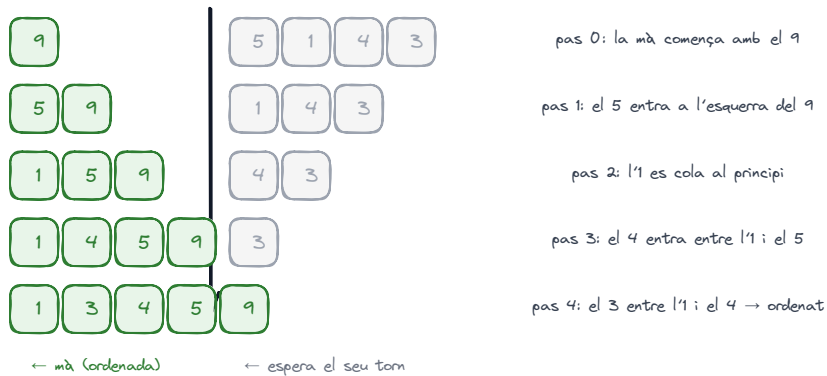
```
for (int i = 1; i < array.length; i++) {  
    int clau = array[i]; // la carta nova  
    int j = i - 1;  
    while (j ≥ 0 && array[j] > clau) {  
        array[j + 1] = array[j]; // desplaça  
        j--;  
    }  
    array[j + 1] = clau; // cau en el seu buit  
}
```



La «mà» (l'esquerra de la barra) **sempre està ordenada**; la resta de l'array espera el seu torn.

La mà de cartes, dibuixada

Inserció: cada carta nova entra en el seu buit dins de la mà ordenada



La clau (la carta nova) es guarda; els majors es desplacen a la dreta i la clau cau en el seu buit • millor cas $O(n)$ amb dades quasi ordenades



Cada element nou s'inserta en el seu lloc. D'ací el nom. Amb {9, 5, 1, 4, 3}: la mà creix

[9] → [5 9] → [1 5 9] → [1 4 5 9] → [1 3 4 5 9] .

Quan és bona la inserció?

Propietat	Valor
Pitjor cas (array invertit)	$O(n^2)$: cada element viatja fins al principi
Millor cas (quasi ordenat)	$O(n)$: una passada de comprovació
Estable?	Sí: manté l'ordre relatiu dels iguals
Memòria extra?	No: ordena in-place

 En la pràctica, és **més ràpida que la bombolla** encara que totes dos siguin $O(n^2)$. De fet, s'usa com a pas final en **TimSort**, l'algoritme que Java usa per defecte en les seues col·leccions.

 **Don Tip:** la inserció és la reina de les dades **quasi ordenades**. Si el teu array té 100 elements i només un parell de despistats, la inserció et sorprendrà.

Comprovació ràpida

?

Per què el bucle interior de la bombolla arriba fins a $n - 1 - i$?

?

Què fa la variable `hiHaIntercanvi`?

?

Bombolla vs inserció: quina diferència hi ha en la manera d'ordenar?

1. Després de cada passada, el major restant ja va quedar col·locat al final: no cal tornar-lo a mirar. · **2.** Detecta si hi va haver intercanvis; si no n'hi va haver cap, fa `break` perquè l'array ja està ordenat. · **3.** La bombolla **intercanvia veïns**; la inserció **col·loca cada element en el seu lloc** desplaçant els majors.

Big O: la tendència, no el cronòmetre

Big O no et diu quants segons tarda un algoritme: et diu **com creix el seu temps quan creixen les dades**. Dos ordinadors poden executar el mateix codi a velocitats diferents — però l'algoritme mana.

Big O: la taxa de creixement, no el cronòmetre



La diferència entre $O(n)$ i $O(n^2)$ amb dades grans és la diferència entre «faig un cafè mentre carrega» i «em jubile abans que acabe».

Un bucle sol $\rightarrow O(n)$ · un bucle dins d'un altre $\rightarrow O(n^2)$ · constants fora, terme dominant es queda

Big O en codi: les tres habituals

ExemplesComplexitat.java

```
// O(1) - CONSTANT
public static int primer(int[] a) {
    return a[0]; // un pas, sempre
}

// O(n) - LINEAL
public static int sumar(int[] a) {
    int s = 0;
    for (int x : a) { s += x; }
    return s; // n passos
}
```

$O(n^2)$ - QUADRÀTICA

```
public static void parells(int[] a) {
    for (int i = 0; i < a.length; i++) {
        for (int j = 0; j < a.length; j++) {
            System.out.println(a[i]
                               + ", " + a[j]);
        }
    }
}
```



Regla del polze: un bucle sol → $O(n)$. Un bucle dins d'un altre → $O(n^2)$.

Les 4 regles d'or de Big O

- 1 **Ignora les constants:** $O(2n)$ és el mateix que $O(n)$. El 2 no importa quan n tendix a l'infinit.
- 2 **Queda't amb el terme dominant:** $O(n^2 + 5n + 1) \rightarrow O(n^2)$. El n^2 es menja la resta.
- 3 **Bucles anidats multipliquen:** un dins d'un altre $\rightarrow n \times n \rightarrow O(n^2)$.
- 4 **Bucles seqüencials sumen:** un i després un altre $\rightarrow O(n + n) \rightarrow O(2n) \rightarrow O(n)$.



Big O descriu el **pitjor cas** (la cota superior). Existixen Big Ω (millor cas) i Big Θ (cas mitjà), però amb Big O tens de sobres. I per a aprovar, també.



Be the Code: l'analista de complexitats

Analisi.java

```
// mètode A
for (int i = 0; i < a.length; i++) { ... }
for (int i = 0; i < a.length; i++) { ... }
// mètode B
for (int i = 0; i < a.length; i++) {
    for (int j = 0; j < a.length; j++) {
        System.out.println(a[i] + " " + a[j]);
    }
}
// mètode C
return a[a.length - 1];
```

Quina és la complexitat de cada mètode?

A → **O(n)**: dos bucles **seqüencials** sumen:
 $O(n + n) = O(2n) = O(n)$.

B → **O(n²)**: dos bucles **anidats**
multipliquen: $n \times n$.

C → **O(1)**: accés directe per índex, sense bucles.

Comprovació ràpida

?

Què mesura exactament Big O?

?

Ordena de menor a major: $O(1)$, $O(n^2)$, $O(n)$, $O(\log n)$, $O(2^n)$.

?

$O(2n)$ i $O(n)$ són el mateix?

1. La **taxa de creixement** del temps quan creix n , no els segons exactes. · 2. $O(1) < O(\log n) < O(n) < O(n^2) < O(2^n)$. · 3. **Sí**: les constants s'ignoren.

Triar l'algoritme adequat

Algoritme	Complexitat	Quan usar-lo?	Quan NO?
Cerca lineal	$O(n)$	Arrays xicotets o desordenats	Arrays grans i ordenats
Cerca binària	$O(\log n)$	Arrays grans ordenats , moltes cerques	Arrays desordenats (escombraria!)
Bombolla	$O(n^2)$	Aprendre; arrays < 50	Qualsevol cosa seriosa
Inserció	$O(n^2)$ / $O(n)$	Arrays xicotets o quasi ordenats	Arrays grans i desordenats

 Per a obrir una nou faries servir una excavadora? Hi ha qui clava una cerca binària en un array de 5 elements, i qui ordena un milió de dades amb bombolla. Els dos estan malament.

La pregunta clau: ordenar abans de buscar?

✓ Ordenes una vegada i busques moltes

Ordena amb alguna cosa decent i després usa binària: la inversió **s'amortitza** amb cada consulta.

✗ **Busques una sola vegada** en un array desordenat

Cerca lineal directa. Ordenar només per a una cerca és **regar el jardí amb xampany**.

i Curiositat: en arrays **molt xicotets** ($< \sim 50$), la lineal sol guanyar fins i tot amb dades ordenades: la sobrecàrrega de la binària no compensa. La teoria importa, però el **context mana**.

Regles d'or per a decidir

- 1 És **xicotet** (< 50)? → Qualsevol val: lineal o inserció per simplicitat.
- 2 És **gran i desordenat**? → Ni bombolla ni inserció. Espera a la U08 (QuickSort, MergeSort).
- 3 És **gran i ordenat**? → Cerca binària, sense pensar-ho.
- 4 Està **quasi ordenat**? → Inserció arrasa: $O(n)$ en la pràctica.
- 5 **Vaig a buscar moltes voltes**? → Ordena bé una vegada i busca amb binària. Una sola? Lineal directa.



REPTE 1: binària a mà, sense mirar

RepBbinaria.java

```
public static int cercaBinaria(  
    int[] array, int objectiu) {  
    // 🧠 EL TEU CODI ACÍ  
    return -1;  
}  
  
// proves = {1,3,5,7,9,11,13,15,17,19}  
// cercaBinaria(proves, 7) → 3  
// cercaBinaria(proves, 8) → -1  
// cercaBinaria(proves, 19) → 9  
// cercaBinaria(new int[]{}, 5) → -1
```

- 1 Declara `esquerra` i `dreta`.
- 2 Escriu el `while` amb la condició correcta.
- 3 Calcula el `mig` anti-desbordament.
- 4 Els tres casos: trobat / major / menor.
- 5 Fora del bucle: torna el «no trobat».



Solució del repte 1

cercaBinaria – solució

```
public static int cercaBinaria(int[] array, int objectiu) {  
    int esquerra = 0;  
    int dreta = array.length - 1;  
    while (esquerra ≤ dreta) {  
        int mig = esquerra + (dreta - esquerra) / 2;  
        if (array[mig] == objectiu) {  
            return mig;  
        } else if (array[mig] < objectiu) {  
            esquerra = mig + 1;  
        } else {  
            dreta = mig - 1;  
        }  
    }  
    return -1;  
}
```



L'error més comú (fins i tot amb 10 anys d'experiència): l'off-by-one. ¿ ≤ o < ? ¿ mig + 1 o mig ?
La resposta: ≤ en la condició, i mig + 1 / mig - 1 en moure els punters. Sense ells, bucle infinit.



EL LÍO: la bombolla que apesta

El departament de qualitat diu que fa mala olor. Quins errors trobes?

BombollaLiosa.java

```
public static void ordenar(int[] arr) {  
    for (int i = 0; i < arr.length; i++) {  
        for (int j = 0; j < arr.length; j++) {  
            if (arr[j] > arr[j + 1]) {  
                int temp = arr[j];  
                arr[j] = arr[j + 1];  
                arr[j + 1] = temp;  
            }  
        }  
    }  
}
```

1. Error d'índexs: el bucle interior va fins a `j < arr.length`; quan `j = arr.length - 1`, accedix a `arr[j + 1] = arr[arr.length]`, que no existix → `ArrayIndexOutOfBoundsException`. Ha d'arribar fins a `arr.length - 1 - i`.

2. Rendiment: sense aprofitar que cada passada col·loca un element al final, i sense el flag `hiHaIntercanvi`: bombolla «sense polir».



Repàs: qui soc?

1 Sóc una recepta per a l'ordinador: finita, precisa i sense ambigüitats.

2 Recórrec l'array un a un. No exigisc ordre, però soc lenta amb dades grans.

3 Obric el diccionari per la meitat i descarte mitja llibre en cada intent.

4 Compare veïns i intercanvie: els grans pugen com bombolles.

5 Ordene com en el pòquer: cada carta nova en el seu buit.

6 No mesure segons: mesure com creix el temps quan creixen les dades.

1. L'algoritme · 2. La cerca lineal · 3. La cerca binària · 4. La bombolla · 5. La inserció · 6. La notació Big O



Repàs: el joc de les decisions

1 Complexitat de la cerca binària? — a) $O(n)$ · b) $O(\log n)$ · c) $O(1)$

2 Passos màxims amb 1.024 elements? — a) 10 · b) 1.024 · c) 11

3 `buscar(new int[] {}, 5)` ben feta? — a) -1 · b) 0 · c) Excepció

4 Quina és $O(n^2)$? — a) Bucle seqüencial · b) Dos anidats · c) `a[0]`

5 Un algoritme $O(n^2)$ tarda 1 s amb 1.000 elements. Amb 2.000? — a) 2 s · b) 4 s · c) 1 s

1. **b** — $O(\log n)$: descarta la meitat en cada pas · 2. **a** — 10: $\log_2(1.024) = 10$ · 3. **a** — -1: el while ni entra ($0 \leq -1$ és false) · 4. **b** — anidats multipliquen · 5. **b** — el doble de dades quadruplica: n^2



Preguntes d'entrevista de treball

1 «Explica'm, com si jo fóra la teua àvia, la diferència entre **cerca lineal** i **binària**.»

2 «Què és la notació **Big O** i per què importa?»

3 «Escriu una **cerca binària a la pissarra**. Ara digues què passa si l'array no està ordenat.»

4 «Quan usaries la **inserció** en comptes de la bombolla?»

5 «Què és l'**off-by-one** i com l'evites en la binària?»

6 «Un $O(n^2)$ tarda 1 s amb 1.000 elements. Quant tardarà amb **10.000**?»

 Preguntes reals per a un programador Java júnior. Si saps respondre les sis, la unitat està apresada — i pots dir « $O(n^2) \rightarrow 100$ segons» sense parpadar.



No hi ha preguntes tontes

? Puc usar `Arrays.sort()` i `Arrays.binarySearch()`?

En programes reals, sí. Ací l'objectiu és **entendre la idea** de davall: com sumar a mà abans d'usar la calculadora. En una entrevista volen veure que ho entens, no que saps importar `java.util.Arrays`.

? Per què «log n» i no «pocs passos»?

Perquè «pocs» no servix per a comparar. El $\log_2$ et diu quantes voltes pots partir entre 2 abans d'arribar a 1. La precisió és el sou del programador.

? Si són totes dos $O(n^2)$, per què la inserció és millor?

Fa **menys intercanvis** (desplaça) i en arrays **quasi ordenats és $O(n)$** de veritat. La bombolla sense flag fa passades senceres vinga o no vinga.

? V o F: «la binària funciona amb qualsevol array, només que a voltes és lenta».

Fals. Amb un array desordenat retorna **resultats incorrectes sense avisar**. No hi ha error: hi ha escombraria silenciosa.

Resum del bloc 2 en 5 frases

- 1 La **bombolla** compara veïns i intercanvia: $O(n^2)$, el flag `hiHaIntercanvi` fa el millor cas $O(n)$.
- 2 La **inserció** col·loca cada element com una carta en la mà ordenada: $O(n)$ amb quasi ordenats, estable i in-place.
- 3 **Big O** mesura la taxa de creixement: constants fora, terme dominant es queda, **anidar multiplica i seqüenciar suma**.
- 4 Triar bé és la mitat de la faena: **mida, ordre inicial i freqüència** de les cerques decideixen.
- 5 Binària i bombolla **a mà**: dos punters i els $+1/-1$ sagrats; dos bucles, intercanvi amb `temp` i flag.

